

topN program.docx

Exported: May 28, 2026

Summary

This Java-based code outlines a **MapReduce framework** designed to process large text datasets to identify the most frequently occurring words. The program begins by using a **Mapper to sanitize and tokenize** raw text, converting strings to lowercase and stripping away punctuation to ensure accurate counting. A **Combiner and Reducer** then aggregate these individual word counts, while a specialized utility class performs a **value-based sort** on the resulting data. Ultimately, the system is engineered to filter these results to provide a **top-ranked list of words**, focusing specifically on those with the **highest number of occurrences** in the source material.

Keywords: MapReduce Program, Top N Occurrences, Word Count Sorting, Hadoop Java API, Mapper and Reducer

Content

Create a Map Reduce program to sort the content in an alphabeticorder listing only top 10 maximum occurrences of words.

```
Driver:import java.io.IOException;import
java.util.StringTokenizer;import      org.apache.hadoop.conf.Configuration;import
org.apache.hadoop.fs.Path;import      org.apache.hadoop.io.IntWritable;import
org.apache.hadoop.io.Text;import      org.apache.hadoop.mapreduce.Job;import
org.apache.hadoop.mapreduce.Mapper;import
org.apache.hadoop.mapreduce.lib.input.FileInputFormat;import
org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;import
org.apache.hadoop.util.GenericOptionsParser;public class TopN {public static void
main(String[]  args)  throws  Exception  {Configuration  conf  =  new
Configuration();String[]  otherArgs  =  (new  GenericOptionsParser(conf,
args)).getRemainingArgs();if (otherArgs.length != 2) {System.err.println("Usage:
TopN      <in>      <out>");System.exit(2);}Job      job      =
Job.getInstance(conf);job.setJobName("Top
N");job.setJarByClass(TopN.class);job.setMapperClass(TopNMapper.class);job.setReducerClass(TopNReducer.class);}
```

```

new Path(otherArgs[0]));FileOutputFormat.setOutputPath(job, new
Path(otherArgs[1]));System.exit(job.waitForCompletion(true) ? 0 : 1);}Mapperpublic
static class TopNMapper extends Mapper<Object, Text, Text, IntWritable> {private
static final IntWritable one = new IntWritable(1);private Text word = new
Text();private String tokens = "[_#$%<>\\^=\\|\\*\\/\\.,\\-:()?!\"'"]";public void
map(Object key, Text value, Mapper<Object, Text, Text,IntWritable>.Context
context) throws IOException, InterruptedException {String cleanLine =
value.toString().toLowerCase().replaceAll(this.tokens, " ");StringTokenizer itr = new
StringTokenizer(cleanLine);while (itr.hasMoreTokens())
{this.word.set(itr.nextToken().trim());context.write(this.word, one);}}}
Combiner:import java.io.IOException;import org.apache.hadoop.io.IntWritable;import
org.apache.hadoop.io.Text;import org.apache.hadoop.mapreduce.Reducer;public
class TopNCombiner extends Reducer<Text, IntWritable, Text, IntWritable> {public
void reduce(Text key, Iterable<IntWritable> values, Reducer<Text, IntWritable,
Text,IntWritable>.Context context) throws IOException, InterruptedException {int
sum = 0;for (IntWritable val : values)sum += val.get();context.write(key, new
IntWritable(sum));}}Reducer:import java.io.IOException;import
java.util.HashMap;import java.util.Map;import
org.apache.hadoop.io.IntWritable;import org.apache.hadoop.io.Text;import
org.apache.hadoop.mapreduce.Reducer;import utils.MiscUtils;public class
TopNReducer extends Reducer<Text, IntWritable, Text, IntWritable> {private
Map<Text, IntWritable> countMap = new HashMap<>();public void reduce(Text key,
Iterable<IntWritable> values, Reducer<Text, IntWritable, Text,IntWritable>.Context
context) throws IOException, InterruptedException {int sum = 0;for (IntWritable val :
values)sum += val.get();this.countMap.put(new Text(key), new IntWritable(sum));}
protected void cleanup(Reducer<Text, IntWritable, Text, IntWritable>.Context
context) throwsIOException, InterruptedException {Map<Text, IntWritable>
sortedMap = MiscUtils.sortByValues(this.countMap);int counter = 0;for (Text key :
sortedMap.keySet()) {if (counter++ == 20)break;context.write(key,
sortedMap.get(key));}}}}MiscUtils.javapackage utils;import java.util.*;public class
MiscUtils {public static <K extends Comparable, V extends Comparable> Map<K,
V> sortByValues(Map<K,V> map) {List<Map.Entry<K, V>> entries = new
LinkedList<Map.Entry<K, V>>(map.entrySet());Collections.sort(entries, new
Comparator<Map.Entry<K, V>>() {@Overridepublic int compare(Map.Entry<K, V>
o1, Map.Entry<K, V> o2) {return o2.getValue().compareTo(o1.getValue());}}});//
LinkedHashMap will keep the keys in the order they are inserted//which is currently
sorted on natural orderingMap<K, V> sortedMap = new LinkedHashMap<K, V>();for
(Map.Entry<K, V> entry : entries) {sortedMap.put(entry.getKey(), entry.getValue());}
return sortedMap;}}

```